arXiv:2408.10161v3 [cs.CV] 3 Aug 2025

NeuFlow-V2: Push High-Efficiency Optical Flow To the Limit

Zhiyong Zhang[1*], Aniket Gupta[2*], Huaizu Jiang[2], Hanumant Singh[1]

***Abstract*— Real-time high-accuracy optical flow estimation is critical for a variety of real-world robotic applications. However, current learning-based methods often struggle to balance accuracy and computational efficiency: methods that achieve high accuracy typically demand substantial processing power, while faster approaches tend to sacrifice precision. These fast approaches specifically falter in their generalization capabilities and do not perform well across diverse real-world scenarios. In this work, we revisit the limitations of the SOTA methods and present NeuFlow-V2, a novel method that offers both — high accuracy in real-world datasets coupled with low computational overhead. In particular, we introduce a novel light-weight backbone and a fast refinement module to keep computational demands tractable while delivering accurate optical flow. Experimental results on synthetic and real-world datasets demonstrate that NeuFlow-V2 provides similar accuracy to SOTA methods while achieving 10x-70x speedups. It is capable of running at over 20 FPS on 512x384 resolution images on a Jetson Orin Nano. The full training and evaluation code is available at `https://github.com/neufieldrobotics/NeuFlow_v2`.**

I. Introduction

Optical flow is the apparent motion of objects, surfaces, or edges in a visual scene caused by the relative movement between the camera and the scene. It represents a dense field of motion vectors that describe how each pixel in an image moves between two consecutive frames. This problem remains unsolved for edge devices which are particularly employed on fast-moving robots like drones. The best algorithms are limited by their generalization capabilities across diverse scenarios, fast-moving objects, occlusions etc.

Optical flow algorithms have seen substantial progress in recent years [1]. Starting from FlowNet [2], learning-based methods for optical flow have shifted towards feature learning for matching, moving away from traditional hand-crafted features like those in Lucas-Kanade [3] or SIFT [4] [5]. Despite these advances, early optical flow methods that rely on CNNs struggle with significant challenges such as handling large displacements [6]. Furthermore, their one-shot architectures do not generalize well to real-world data [7] [2], [6], [8], [9]. This can be primarily attributed to the difficulty in collecting ground truth optical flow data in the real-world, simulation is predominantly used to generate sufficient training data [2], [10]. However, training with simulation data can lead to overfitting due to unrealistic illumination, reflections, and monotonous scenes [11], [12].

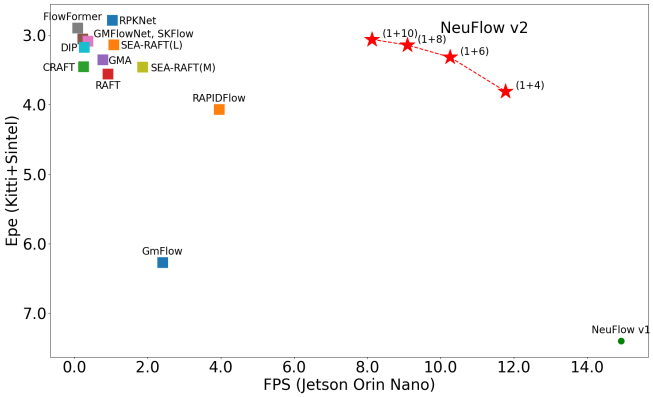

Fig. 1: End Point Error (EPE) of KITTI and Sintel datasets vs. Frames Per Second (FPS) throughput on an edge computing platform (Jetson Orin Nano). Individual points represent a broad class of optical flow methods. Our algorithm is comparable in accuracy but significantly more efficient, approaching an order of magnitude improvement in computational complexity. All models were trained solely on the FlyingThings datasets. For the notation "NeuFlow-V2 $(x + y)$", x represents number of refinement iterations at 1/16th scale and y represents number of refinement iterations at 1/8th scale.

Starting with RAFT [7], iterative refinements have partially mitigated the generalization issue while also capturing larger motions [13], [14] albeit with an increased computational cost [15], [16]. Recent research has further improved accuracy and generalization by incorporating the latest modules, such as Transformer [17], Partial Kernel Convolution [18], Super Kernel [19] etc. However, these methods are generally more computation heavy due to the iterative refinement process. Some models need over 30 iterations to generate a stable optical flow [7], while others reduce the number of iterations but increase the computational load of each iteration [18], [20].

NeuFlow-V1 [21] provides real-time inference speeds on edge devices like Jetson Orin Nano for optical flow estimation. However, it faces notable challenges in generalizing to real-world datasets. Although incorporating refinement modules from prior works [15], [13] in the NeuFlow-V1 architecture could address these issues, they significantly increase computational overhead, limiting real-time feasibility. To overcome this, we present an efficient iterative refinement module that leverages only CNN layers arranged in a recurrent fashion. This helps mitigate the generaliza-

[1]Department of Electrical and Computer Engineering, Northeastern University, Boston, MA 02115.
[2]Khoury College of Computer Sciences, Northeastern University, Boston, MA 02115.
*Equal contribution.
Huaizu Jiang is supported by the National Science Foundation under Award IIS-2310254.
{zhang.zhiyo, gupta.anik, h.jiang, ha.singh}@northeastern.edu

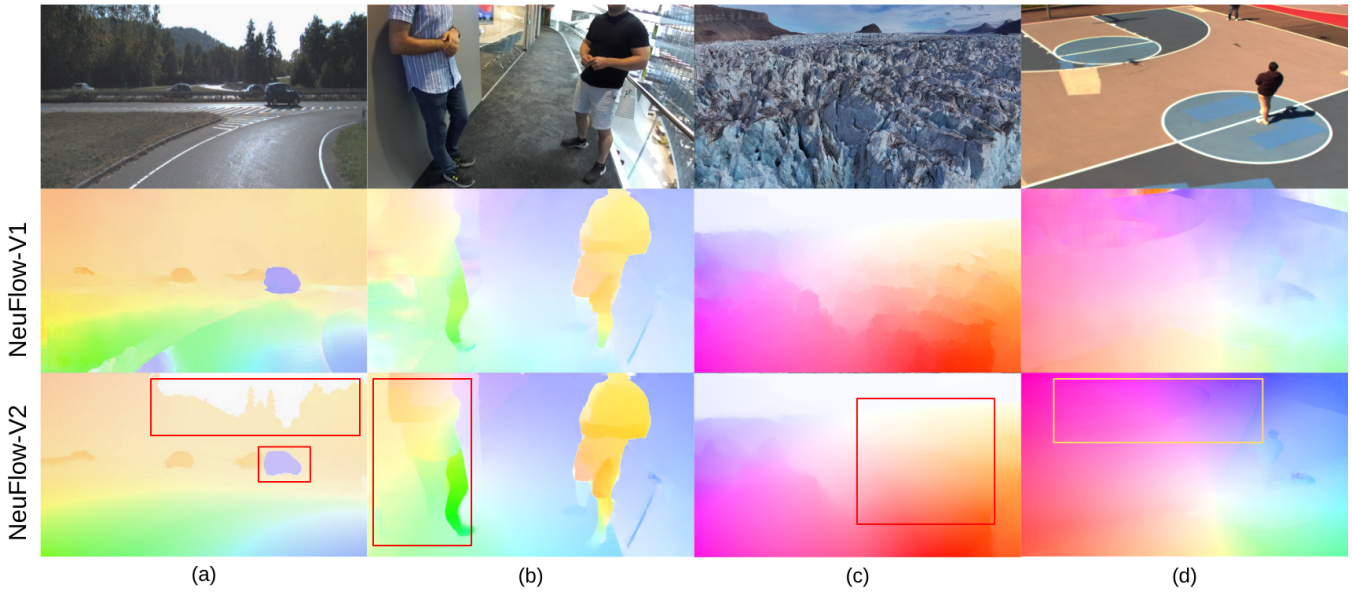

Fig. 2: Optical flow results on unseen real-world images. In comparison to NeuFlow-V1, NeuFlow-V2 outputs more accurate optical flow at capturing the scene details (highlighted with bounding boxes in (a) and (b)) and also much smoother and more accurate results in (c) and (d).

tion issue while maintaining fast inference speed on edge devices. To further optimize the NeuFlow-V1 architecture, we introduce a new backbone that fuses multi-scale features more effectively, achieving a 24% (backbone inference time) speedup compared to the original design while improving accuracy.

We evaluate NeuFlow-V2 extensively on both synthetic [22], [10] and real-world [23] datasets. NeuFlow-V2 achieves real-time inference while delivering near state-of-the-art accuracy on edge devices, making it well-suited for deployment on small robotic platforms. Fig. 1 plots the end point error (EPE) vs FPS for the latest optical flow methods. NeuFlow-V2 significantly outperforms NeuFlow-V1 while still maintaining real-time speeds. Fig. 2 shows generalization examples of NeuFlow-V2 on unseen real-world data in comparison with NeuFlow-V1.

In summary, we make the following two critical contributions in devising NeuFlow-V2.

- Lightweight Backbone: A simple CNN-based backbone for extracting low-level features from multi-scale images. Unlike commonly used architectures such as ResNet [24] or Feature Pyramid Networks [25], this efficient backbone is sufficient for accurate optical flow estimation.
- Efficient Iterative Refinement Module: A lightweight recurrent module that refines optical flow predictions while maintaining efficiency. Instead of computationally expensive approaches like LSTMs [26] or GRUs [27], our simplified refinement achieves higher accuracy with minimal overhead.

II. Related Work

FlowNet [2] was the first deep learning-based optical flow estimation method, introducing two variants: FlowNetS and FlowNetC, along with the synthetic FlyingChairs dataset for end-to-end training and benchmarking. An improved version, FlowNet 2.0 [8], fused cascaded FlowNets with a small displacement module, decreasing the estimation error by more than 50% while being marginally slower.

Following FlowNet 2.0 [8], researchers developed more lightweight optical flow methods. SPyNet [9] is 96% smaller than FlowNet in terms of model parameters. PWC-Net [6] is 17 times smaller than FlowNet 2. LiteFlowNet [28] is 30 times smaller in model size and 1.36 times faster in running speed compared to FlowNet 2. LiteFlowNet 2 [29] improved optical flow accuracy on each dataset by around 20% while being 2.2 times faster. LiteFlowNet 3 [30] further enhanced flow accuracy. RapidFlow [31] combines efficient NeXt1D convolution blocks with a fully recurrent structure to decrease computational costs. DCVNet [32] proposes constructing cost volumes with different dilation factors to capture small and large displacements simultaneously. NeuFlow-V1[21], our previous work, is a fastest optical flow method, being over ten times faster than mainstream optical flow methods while maintaining comparable accuracy on the Sintel and FlyingThings datasets.

More recently, RAFT [7] used recurrent all-pairs field transforms to achieve strong cross-dataset generalization. Following RAFT, GMA [14] introduced a global motion aggregation module to improve estimation in occluded regions. GMFlow [13] reformulated optical flow as a global matching problem. GMFlowNet [17] efficiently performed global matching by applying argmax on 4D cost volumes. CRAFT [15] used a Semantic Smoothing Transformer layer to make features more global and semantically stable. FlowFormer [16], [33] encodes the 4D cost tokens into a cost memory with alternate-group transformer layers in a latent space. SKFlow [19] benefits from super kernels to com-

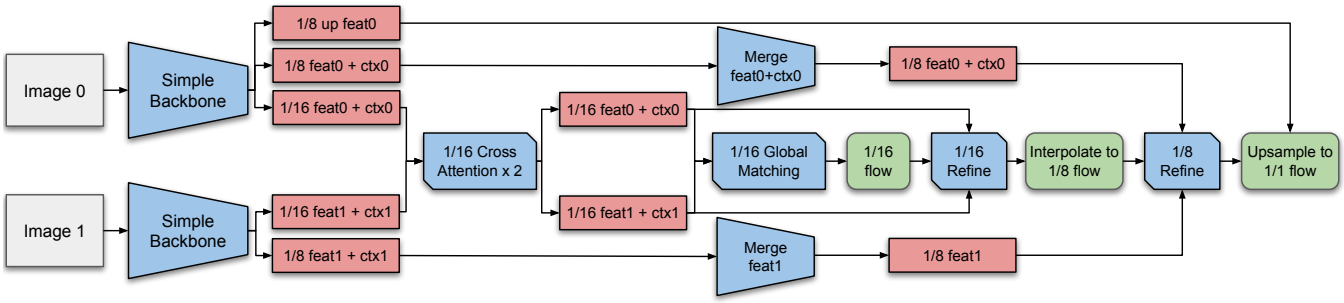

Fig. 3: NeuFlow-V2 architecture. We begin with a simple CNN backbone that outputs features and context at 1/8 and 1/16 scales for both images. The features at the 1/16 scale are then fed into cross-attention layers for feature enhancement. Next, we perform global matching to obtain an initial flow at the 1/16 scale, which is refined through one iteration. This flow is upsampled to a 1/8 scale and further refined over eight iterations. The refined 1/8-scale flow is then upsampled to full resolution using a convex upsampling module.

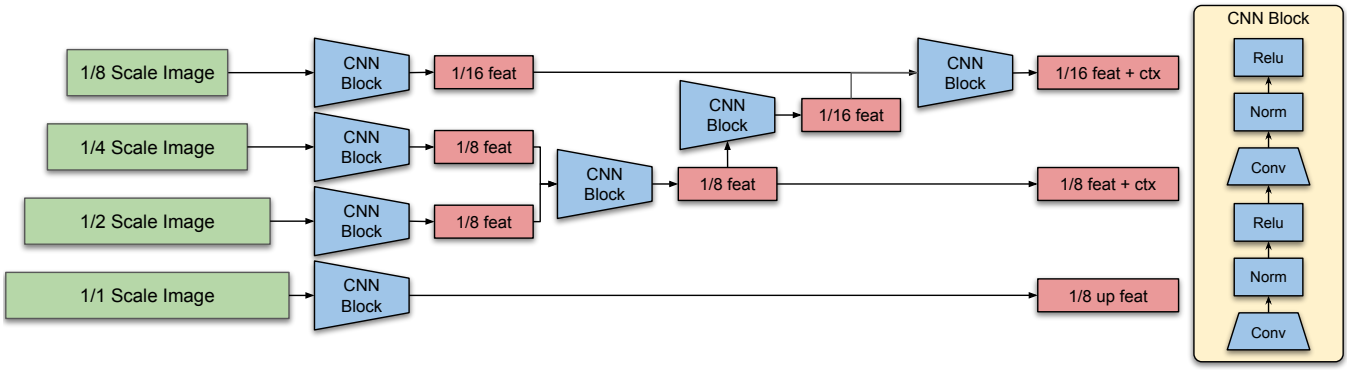

Fig. 4: NeuFlow-V2 simple backbone. We downsample the image into various scales, ranging from 1/1 to 1/8. A simple CNN block, consisting of two [Convolution, Norm, ReLU] layers, is used to extract low-level features from the image at various scales. Then, using the same CNN block design, we merge and resize these features into 1/8 and 1/16 scale outputs. The backbone outputs 1/8 and 1/16-scale features and context for further flow estimation, along with an additional 1/8-scale feature for convex upsampling.

plement the absent matching information and recover the occluded motions. DIP [34] introduced the first end-to-end PatchMatch-based method, achieving high-precision results with lower memory. RPKNet [18] utilized Partial Kernel Convolution layers to produce variable multi-scale features and efficient Separable Large Kernels to capture large context information. Sea-Raft [20] proposed a new loss (mixture of Laplace) and directly regressed an initial flow for faster convergence. Many works have also been proposed to either reduce computational costs or improve flow accuracy [35].

III. Proposed Approach: NeuFlow v2

We introduce NeuFlow-V2, an enhanced version of NeuFlow-V1 that preserves real-time inference speed on edge devices while achieving near state-of-the-art accuracy on real-world datasets. To accomplish this, we propose two key innovations: (1) a fast and efficient backbone that optimally balances inference speed and accuracy, and (2) a lightweight iterative refinement module that significantly improves NeuFlow-V2's generalization to real-world datasets. The overall architecture of our model is illustrated in Fig. 3.

A. Backbone

As presented in [21], a shallow backbone extracting low-level features is sufficient for computing pixel similarity and computing optical flow. We build on this insight and propose our simple backbone architecture. As presented in Fig. 4 our backbone only extracts features from 1/2th, 1/4th and 1/8th images using a CNN block composed of convolution, normalization, and ReLU layers. This same CNN block is used to concatenate and resize these features into the desired output scale, specifically to 1/16-scale features and context, as well as 1/8-scale features and context. Features are used for correlation computation, while context is used for flow refinement.

Note that the 1/1-scale image is used solely for convex upsampling. In our experiments, we find that extracting features from 1/1-scale images leads to overfitting on the training data (FlyingThings) and thus are ineffective in flow computation on unseen data (Sintel, KITTI). See IV-C for details.

B. Cross-Attention And Global Matching

Cross-attention is used to exchange information between images globally, enhancing the distinctiveness of matching

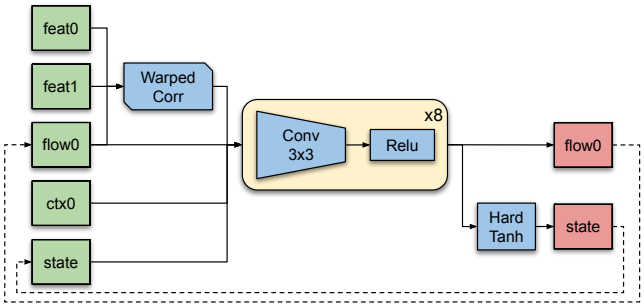

Fig. 5: NeuFlow-V2 iterative refinement. We first compute the correlation within nearby pixels and warp these values using the currently estimated flow. The warped correlation, current estimated flow, context features, and hidden state are then fed into a series of 3×3 convolution layers followed by ReLU activation, repeated eight times. At the end of these layers, the network outputs both the refined flow and an updated hidden state for the next iteration. Instead of using GRU or LSTM modules, we use a simpler recurrent design without any gates. The `HardTanh` function is applied to update the hidden state, which mitigates the saturation issue related to the `tanh` function.

features and reducing the similarity of unmatched features. Global matching is then applied to find corresponding features globally, enabling the model to handle large pixel displacements, such as in fast-moving camera situations. To reduce the computational burden of cross-attention and global matching, we operate on 1/16-scale features instead of the 1/8-scale.

Similar to NeuFlow-V1 and GMFlow+ [36], we utilize Transformers to implement cross-attention and global matching. While flow self-attention is typically used to refine pixels based on self-similarity after global matching [36], [13], we instead use our refinement module to iteratively refine the estimated flow.

C. Iterative Refinement Module

We first compute the correlation within nearby 9×9 neighborhoods and warp this correlation using the estimated flow. We then concatenate the warped correlation, context features, estimated flow, and previous hidden state, processing them through eight layers of 3×3 convolutional layers followed by ReLU activation to output the refined optical flow and updated hidden state. Refinement is performed for the flow at both the 1/16 and 1/8 scales. Fig. 5 illustrates the architecture of our refinement module.

To address the vanishing or exploding gradient problem, most RNNs use GRU or LSTM modules to compute the current hidden state based on the previous hidden state and current inputs (warped correlation, context, and flow) and to decode the estimated flow using the current hidden state. However, GRU and LSTM may be computationally heavy in a real-time method. In our design, we adopt a simpler design without any gates in a recurrent model, where we only have a set of 3×3 convolution layers followed by ReLU.

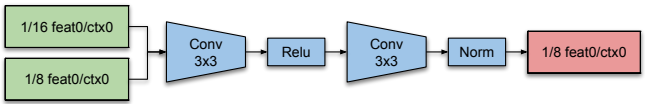

Fig. 6: NeuFlow-V2 merge module. We concatenate the 1/8-scale features/context with the interpolated 1/16-scale features/context and use simple CNN blocks to output 1/8-scale features/context, thereby incorporating both global and local information.

The `tanh` function is commonly applied to the output hidden state, keeping values within the range (-1,1). However, when hidden state values approach these limits, the corresponding inputs to `tanh` in the previous layer can become extremely large or small, leading to numerical overflow. To mitigate the gradient vanishing/explosion issue related to the saturation of the `tanh` activation function, we use the `HardTanh` [37] for the hidden state update. We clip the input to be in the range of [-4, 4], where there are reasonable gradients values. As shown in the ablation study in Section IV-C, our simple design does not only run faster but also produces more accurate optical flow than using GRU.

D. Multi-Scale Feature/Context Merge

Our simple backbone lacks the depth of convolution, resulting in features having a small receptive field. Cross-attention at the 1/16-scale provides global attention, offering a global feature/context. However, 1/8-scale features/context do not have a global receptive field. Therefore, we merge the 1/16-scale global features/context with the 1/8-scale local features/context to ensure that the 1/8-scale features/context contain both global and local information.

The merge block (Fig. 6) consists of two layers of CNNs with ReLU activation and normalization. In practice, features and context are merged individually using the same merge block structure.

IV. Experiments

A. Training and Evaluation Datasets

We first train the model solely on the FlyingThings [10] dataset for a fair comparison with other models, as most models have undergone the same procedure. For evaluation, we followed the common practice of using the Sintel and KITTI datasets to demonstrate the model's generalization capabilities. We followed the same procedure and data augmentation settings as RAFT, utilizing RAFT's [7] training and evaluation code.

B. Comparison with Latest Optical Flow Methods

We compare our method against several state-of-the-art optical flow methods, including methods optimized for highest accuracy (Slow) and methods optimized for highest speed (Fast) in Tab. I. To show the effectiveness of our approach across GPU platforms, we measure the computation time

	Method	Sintel (train)		RTX 2080 (s)	Jetson Orin Nano (s)	Batch Size (8G)	KITTI-15		RTX 2080 (s)	Jetson Orin Nano (s)	Batch Size (8G)
		clean	final				EPE	F1			
Slow	SKFlow (32 iters)	1.22	2.46	0.365	4.181	17	4.27	15.5	0.408	4.478	16
	SEA-RAFT(M) (4 iters)	1.21	4.04	**0.061**	**0.524**	23	4.29	14.2	**0.075**	**0.549**	22
	DIP (20 iters)	1.30	2.82	0.499	3.615	12	4.29	13.7	0.523	3.767	12
	GMFlowNet (32 iters)	1.14	2.71	0.227	2.626	12	4.24	15.4	0.231	2.761	12
	FlowFormer	**1.01**	**2.40**	1.007	11.196	4	4.09	14.7	1.002	11.172	5
	RPKNet (12 iters)	1.12	2.45	0.158	0.947	**68**	3.79	13.0	0.157	0.976	**64**
	SEA-RAFT(L) (12 iters)	1.19	4.11	0.096	0.910	22	**3.62**	**12.9**	0.101	0.953	20
	GMA (32 iters)	1.30	2.74	0.152	1.245	17	4.69	17.1	0.161	1.343	16
	RAFT (32 iters)	1.43	2.71	0.125	1.060	24	5.04	17.4	0.129	1.126	23
	CRAFT (32 iters)	1.27	2.79	0.347	N/A	2	4.88	17.5	0.385	N/A	2
Fast	RAPIDFlow (12 iters)	1.58	2.94	0.038	0.252	103	5.87	17.7	0.045	0.254	99
	GMFlow (1 iter)	1.50	2.96	0.046	0.404	24	10.3	33.6	0.055	0.426	22
	NeuFlow-V1	1.66	3.13	**0.009**	**0.064**	**107**	12.4	32.5	**0.010**	**0.070**	**115**
	NeuFlow-V2(9 iters)	**1.24**	**2.67**	0.015	0.106	26	**4.33**	**15.3**	0.015	0.114	21

TABLE I: This table compares the latest optical flow methods based on their highest accuracy. All models were trained on the FlyingThings dataset and evaluated on the Sintel and KITTI training sets. Inference time was measured on both an RTX 2080 and an edge computing device, the Jetson Orin Nano, using half-precision models. Batch size was determined during batch inference and both RTX 3080 and Jetson Nano used have 8GB memory. CRAFT is marked as N/A because the Jetson Orin Nano crashed during inference. The table shows that NeuFlow-V2 achieves the best accuracy in the Fast methods while still running in real-time. This table also highlights that NeuFlow-V1 while being really fast, does not work well on real-world KITTI data. *Best performance is marked in bold and second best is marked with underline

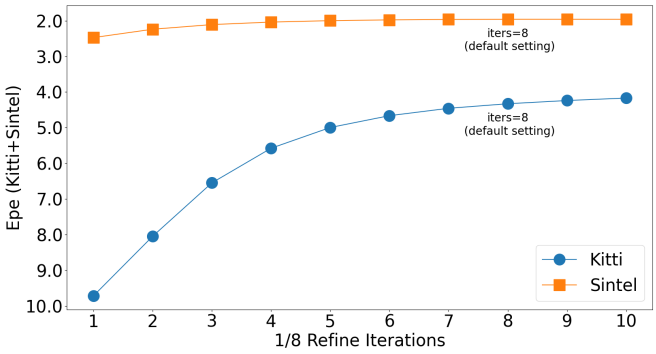

Fig. 7: End Point Error (EPE) for the KITTI and Sintel datasets vs. the number of 1/8 refinement iterations: The default configuration is 8 iterations. Accuracy on Sintel converges quickly within the first few iterations, while KITTI continues to improve until 8-10 iterations.

on Nvidia RTX 2080 GPU and the edge computing device Nvidia Jetson Orin Nano (8GB). We evaluate on the Sintel dataset with 1024×436 resolution and KITTI-15 dataset with 1242×375 resolution. The inference batch size is measured on the Jetson Orin Nano with 8GB memory to assess memory usage of all methods.

Tab. I shows that amongst all the methods NeuFlow-V2 achieves comparable performance to SEA-RAFT(M) [20] and SKFlow [19] while being 5x-40x faster on the Sintel Dataset. On the KITTI dataset, NeuFlow-V2 achieves comparable performance to SKFlow[19], SEA-RAFT(M)[20], DIP[34] and FlowFormer[16] while being 5x-110x faster. Amongst fast methods, NeuFlow-V2 achieves the best accuracy on both datasets while being 2.5x faster than RAPIDFlow[31] and 4x faster than GMFlow[13]. In comparison to NeuFlow-V1, we achieve a huge jump in accuracy on the real-world KITTI dataset and significant improvements on the Sintel dataset - which justifies the choices we have made in designing NeuFlow-V2 over NeuFlow-V1.

C. Ablation Study

Backbone Module: We found that using full scale features in the backbone actually does not help in estimating the 1/8th scale optical flow and overfits on the training dataset (Flying Things) leading to a slight drop in performance on both Sintel and KITTI datasets. Table II shows this drop in performance when we add full scale features in the backbone. Moreover, using the YOLO v8 and the NeuFlow-V1 backbone also lead to the same effect of overfitting on the training dataset and thus poor generalization on real-world data.

Refine Module: We use 8 layers of CNN to output both refined optical flow and the hidden state in the refinement module. We experimented by reducing and adding 2 layers to observe the impact. The results show that reducing the number of layers slightly decreases accuracy, while adding layers does not improve accuracy. This indicates that eight layers provide a balanced configuration.

Our default feature dimensions are 128 for the 1/16 refinement and 96 for the 1/8 refinement. Reducing the feature dimensions by half (64 for the 1/16 refinement and 48 for the 1/8 refinement) results in a significant drop in accuracy.

In the refinement module, we use multiple layers of CNN to output the hidden state. We also experimented with replacing the first CNN layer with a ConvGRU to output the hidden state and using the remaining seven CNN layers to decode a refined flow. But including ConvGRU increases

Method	Things		Sintel (train)		Jetson Orin Nano (s)	KITTI-15		Jetson Orin Nano (s)
	train	val	clean	final		EPE	F1	
Full	2.97	3.44	1.24	2.67	0.106	4.33	15.3	0.114
Backbone Module								
YOLO v8 Backbone	2.56	3.15	1.34	2.96	0.111	4.87	17.0	0.121
1/1 backbone	2.73	3.28	1.19	2.81	0.112	4.92	15.7	0.122
NeuFlow-V1 backbone	2.77	3.34	1.24	2.87	0.116	4.98	15.8	0.128
Refine Module								
-2 layers	2.85	3.41	1.22	2.76	0.099	5.21	16.5	0.106
+2 layers	2.67	3.17	1.18	2.80	0.113	4.62	15.6	0.122
half feature dimension	3.26	3.79	1.37	2.91	0.095	6.66	21.7	0.102
use ConvGRU	3.00	3.62	1.28	3.87	0.121	7.36	20.4	0.129
Architecture								
w/o cross attention	3.99	4.37	1.60	4.17	0.098	5.18	16.3	0.105
w/o global match, 1/16 refine=4	2.85	3.24	1.21	2.88	0.111	4.89	16.1	0.120
w/o 1/16 refine	3.00	3.79	1.40	4.06	0.103	5.52	19.6	0.111

TABLE II: Ablation Study: Starting with ablations on the backbone, we remove the full scale features from the backbone used in NeuFlow-V1 as it leads to slightly better performance. We also experiment with adding/reducing the number of layers in the Iterative refinement module and find out that setting 8 refinement layers provides a good balance in accuracy and computation time. Removing Cross-attention or global matching as expected leads to drop in accuracy. We also find out that removing refinement on 1/16 scale also leads to a significant drop in accuracy

Refinement iterations	Things		Sintel (train)		Jetson Orin Nano (s)	KITTI-15		Jetson Orin Nano (s)
	train	val	clean	final		EPE	F1	
1/16 iters=1, 1/8 iters=8	**2.97**	**3.44**	**1.24**	**2.67**	**0.106**	**4.33**	**15.3**	**0.114**
1/16 iters=3, 1/8 iters=8	2.87	3.35	1.22	2.80	0.111	4.24	15.3	0.119
1/16 iters=5, 1/8 iters=8	2.86	3.37	1.22	2.82	0.116	4.17	15.2	0.124
1/16 iters=1, 1/8 iters=4	3.14	3.63	1.31	2.76	0.082	5.58	17.9	0.088
1/16 iters=1, 1/8 iters=6	3.02	3.50	1.26	2.69	0.094	4.66	15.9	0.101
1/16 iters=1, 1/8 iters=8	**2.97**	**3.44**	**1.24**	**2.67**	**0.106**	**4.33**	**15.3**	**0.114**
1/16 iters=1, 1/8 iters=10	2.95	3.41	1.23	2.68	0.119	4.17	15.0	0.127

TABLE III: This table shows how different iterations affect both accuracy and inference time. The default configuration is 1 iteration of 1/16 refinement and 8 iterations of 1/8 refinement.

the computation burden by 20% for the same number of iterations while being worse in performance.

Architecture: Cross-attention is used to exchange information between two input images globally. Removing it does not significantly affect accuracy on the KITTI dataset, but it causes a substantial drop in accuracy on the Sintel dataset.

Global matching provides the initial optical flow, which can handle large motions. Removing it and adding three more iterations of 1/16 refinement helps address large motion issues. The drop in accuracy indicates that global matching is working efficiently without requiring multiple refinements.

We only perform one iteration of 1/16 refinement. If we remove this and rely entirely on 1/8 refinement, overfitting occurs, as the training set accuracy remains unchanged while the accuracy drops significantly on all validation sets.

Different Iterations: The default iteration count for 1/16 refinement is set to one, as additional iterations do not significantly improve accuracy. In contrast, 1/8 refinement benefits from more iterations. The default eight iterations already provide decent accuracy, but adding more iterations can further improve accuracy at the cost of increased inference time. See Fig. 7 for more details.

V. Conclusions and Future Work

In this paper, we introduced NeuFlow-V2 , an efficient optical flow estimation framework that significantly enhances real-time performance without sacrificing accuracy. Our approach achieves competitive results while operating up to ten times faster than many existing methods, making it particularly well-suited for deployment on edge computing devices. The innovative design of our network—incorporating a lightweight backbone and an efficient iterative refinement module—demonstrates that high-quality optical flow estimation can be both efficient and practical for real-world applications. Comprehensive experiments validate the effectiveness of our method across various benchmarks, underscoring its potential for dynamic environments and resource-constrained settings. And more importantly, it paves the way for future research and practical applications. Looking ahead, we plan to further optimize memory efficiency and reduce parameter counts, thereby enhancing the model's adaptability and scalability without compromising performance.

REFERENCES

[1] M. Zhai, X. Xiang, N. Lv, and X. Kong, "Optical flow and scene flow estimation: A survey," *Pattern Recognition*, vol. 114, p. 107861, 2021.

[2] A. Dosovitskiy, P. Fischer, E. Ilg, P. Hausser, C. Hazirbas, V. Golkov, P. Van Der Smagt, D. Cremers, and T. Brox, "Flownet: Learning optical flow with convolutional networks," in *Proceedings of the IEEE international conference on computer vision*, 2015, pp. 2758–2766.

[3] B. D. Lucas and T. Kanade, "An iterative image registration technique with an application to stereo vision," in *IJCAI'81: 7th international joint conference on Artificial intelligence*, vol. 2, 1981, pp. 674–679.

[4] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *International journal of computer vision*, vol. 60, pp. 91–110, 2004.

[5] C. Liu, J. Yuen, A. Torralba, J. Sivic, and W. T. Freeman, "Sift flow: Dense correspondence across different scenes," in *Computer Vision–ECCV 2008: 10th European Conference on Computer Vision, Marseille, France, October 12-18, 2008, Proceedings, Part III 10*. Springer, 2008, pp. 28–42.

[6] D. Sun, X. Yang, M.-Y. Liu, and J. Kautz, "Pwc-net: Cnns for optical flow using pyramid, warping, and cost volume," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 8934–8943.

[7] Z. Teed and J. Deng, "Raft: Recurrent all-pairs field transforms for optical flow," in *Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part II 16*. Springer, 2020, pp. 402–419.

[8] E. Ilg, N. Mayer, T. Saikia, M. Keuper, A. Dosovitskiy, and T. Brox, "Flownet 2.0: Evolution of optical flow estimation with deep networks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 2462–2470.

[9] A. Ranjan and M. J. Black, "Optical flow estimation using a spatial pyramid network," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 4161–4170.

[10] N. Mayer, E. Ilg, P. Hausser, P. Fischer, D. Cremers, A. Dosovitskiy, and T. Brox, "A large dataset to train convolutional networks for disparity, optical flow, and scene flow estimation," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 4040–4048.

[11] W. Wang, D. Zhu, X. Wang, Y. Hu, Y. Qiu, C. Wang, Y. Hu, A. Kapoor, and S. Scherer, "Tartanair: A dataset to push the limits of visual slam," in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2020, pp. 4909–4916.

[12] L. Mehl, J. Schmalfuss, A. Jahedi, Y. Nalivayko, and A. Bruhn, "Spring: A high-resolution high-detail dataset and benchmark for scene flow, optical flow and stereo," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 4981–4991.

[13] H. Xu, J. Zhang, J. Cai, H. Rezatofighi, and D. Tao, "Gmflow: Learning optical flow via global matching," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 8121–8130.

[14] S. Jiang, D. Campbell, Y. Lu, H. Li, and R. Hartley, "Learning to estimate hidden motions with global motion aggregation," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2021, pp. 9772–9781.

[15] X. Sui, S. Li, X. Geng, Y. Wu, X. Xu, Y. Liu, R. Goh, and H. Zhu, "Craft: Cross-attentional flow transformer for robust optical flow," in *Proceedings of the IEEE/CVF conference on Computer Vision and Pattern Recognition*, 2022, pp. 17 602–17 611.

[16] Z. Huang, X. Shi, C. Zhang, Q. Wang, K. C. Cheung, H. Qin, J. Dai, and H. Li, "Flowformer: A transformer architecture for optical flow," in *European conference on computer vision*. Springer, 2022, pp. 668–685.

[17] S. Zhao, L. Zhao, Z. Zhang, E. Zhou, and D. Metaxas, "Global matching with overlapping attention for optical flow estimation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 17 592–17 601.

[18] H. Morimitsu, X. Zhu, X. Ji, and X.-C. Yin, "Recurrent partial kernel network for efficient optical flow estimation," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 5, 2024, pp. 4278–4286.

[19] S. Sun, Y. Chen, Y. Zhu, G. Guo, and G. Li, "Skflow: Learning optical flow with super kernels," *Advances in Neural Information Processing Systems*, vol. 35, pp. 11 313–11 326, 2022.

[20] Y. Wang, L. Lipson, and J. Deng, "Sea-raft: Simple, efficient, accurate raft for optical flow," *arXiv preprint arXiv:2405.14793*, 2024.

[21] Z. Zhang, H. Jiang, and H. Singh, "Neuflow: Real-time, high-accuracy optical flow estimation on robots using edge devices," *arXiv preprint arXiv:2403.10425*, 2024.

[22] D. J. Butler, J. Wulff, G. B. Stanley, and M. J. Black, "A naturalistic open source movie for optical flow evaluation," in *Computer Vision–ECCV 2012: 12th European Conference on Computer Vision, Florence, Italy, October 7-13, 2012, Proceedings, Part VI 12*. Springer, 2012, pp. 611–625.

[23] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? the kitti vision benchmark suite," in *2012 IEEE conference on computer vision and pattern recognition*. IEEE, 2012, pp. 3354–3361.

[24] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.

[25] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 2117–2125.

[26] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[27] K. Cho, B. Van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using rnn encoder-decoder for statistical machine translation," *arXiv preprint arXiv:1406.1078*, 2014.

[28] T.-W. Hui, X. Tang, and C. C. Loy, "Liteflownet: A lightweight convolutional neural network for optical flow estimation," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 8981–8989.

[29] ——, "A lightweight optical flow cnn—revisiting data fidelity and regularization," *IEEE transactions on pattern analysis and machine intelligence*, vol. 43, no. 8, pp. 2555–2569, 2020.

[30] T.-W. Hui and C. C. Loy, "Liteflownet3: Resolving correspondence ambiguity for more accurate optical flow estimation," in *Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part XX 16*. Springer, 2020, pp. 169–184.

[31] H. Morimitsu, X. Zhu, R. M. Cesar-Jr, X. Ji, and X.-C. Yin, "Rapidflow: Recurrent adaptable pyramids with iterative decoding for efficient optical flow estimation," 2024.

[32] H. Jiang and E. Learned-Miller, "Dcvnet: Dilated cost volume networks for fast optical flow," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2023, pp. 5150–5157.

[33] X. Shi, Z. Huang, D. Li, M. Zhang, K. C. Cheung, S. See, H. Qin, J. Dai, and H. Li, "Flowformer++: Masked cost volume autoencoding for pretraining optical flow estimation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 1599–1610.

[34] Z. Zheng, N. Nie, Z. Ling, P. Xiong, J. Liu, H. Wang, and J. Li, "Dip: Deep inverse patchmatch for high-resolution optical flow," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 8925–8934.

[35] L. Kong, C. Shen, and J. Yang, "Fastflownet: A lightweight network for fast optical flow estimation," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 10 310–10 316.

[36] H. Xu, J. Zhang, J. Cai, H. Rezatofighi, F. Yu, D. Tao, and A. Geiger, "Unifying flow, stereo and depth estimation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2023.

[37] R. Collobert, J. Weston, L. Bottou, M. Karlen, K. Kavukcuoglu, and P. Kuksa, "Natural language processing (almost) from scratch," *Journal of machine learning research*, vol. 12, pp. 2493–2537, 2011.